SSRF with Blacklist-Based Input Filter

Platform: PortSwigger Web Security Academy
Vulnerability: Server-Side Request Forgery (SSRF)
Security Mechanism: Blacklist-Based Input Filtering
Lab Status: Practical Lab Write-up

1. Overview

Server-Side Request Forgery occurs when an application makes server-side requests to destinations that can be influenced by user input. If these requests are not properly restricted, an attacker may access internal services or administrative resources.

In this lab, the application attempts to prevent SSRF by using a blacklist-based input filter. Such filters block known dangerous hostnames, IP addresses, or URL paths but may fail to recognize alternative representations of the same destination.

2. Lab Objective

The objective is to bypass the application's blacklist-based SSRF filter and access the restricted internal administrative functionality within the lab environment.

3. Methodology

1. Open the lab application and locate the product stock-checking feature.
2. Intercept a stock-check request using Burp Suite Proxy.
3. Examine the URL parameter used by the server to retrieve stock information.
4. Test the application's filtering behavior by observing how it handles requests to restricted internal destinations.
5. Determine which hostnames, IP-address representations, URL paths, or encoded characters are rejected by the filter.
6. Investigate whether an alternative representation of the same internal destination passes the blacklist checks.
7. Test the behavior of URL encoding and destination parsing in the authorized lab, paying attention to differences between filter validation and server-side URL interpretation.
8. Verify whether the modified request reaches the intended restricted resource and satisfies the lab objective.

4. Technical Explanation

A blacklist-based filter blocks a predefined collection of strings or patterns associated with potentially dangerous destinations. This approach is often fragile because URLs can be represented in multiple ways.

Depending on the implementation, potential weaknesses include:

- Alternative textual representations of loopback addresses.
- Encoded characters in URL paths.
- Inconsistent URL normalization.
- Differences between the filter's parsing logic and the HTTP client's interpretation.
- Incomplete restrictions on internal IP ranges or administrative paths.

If the filter and the component making the request interpret the same input differently, an attacker may bypass the filter.

5. Security Impact

A successful bypass may allow an attacker to:

- Access internal administrative endpoints.
- Retrieve information from internal services.
- Reach resources that are intended to be inaccessible to external users.
- Expand the attack surface of the application through server-side network access.

The severity depends on which internal services are reachable and what actions they permit.

6. Remediation

- Prefer a strict allowlist of approved destinations instead of a blacklist.
- Parse URLs with a reliable, consistent URL parser.
- Normalize and validate hostnames, paths, and IP addresses before making requests.
- Resolve DNS names and verify that the resulting IP addresses are permitted.
- Block loopback, private, link-local, and other restricted address ranges where appropriate.
- Revalidate every redirect destination.
- Apply outbound network controls so application-level filter bypasses cannot provide unrestricted internal access.

7. Conclusion

This lab demonstrates the limitations of blacklist-based SSRF protection. Blocking a few known strings is not a reliable way to control server-side requests because alternative representations

and parsing differences may allow restrictions to be bypassed. Strict destination allowlisting and network-level controls provide stronger protection.

Reference: PortSwigger Web Security Academy — Server-side request forgery (SSRF)

https://portswigger.net/web-security/ssrf